SWCON221 Microservice Programming

Lecture 6

Docker Overview

2026. 9. 1

Sungwon Lee

Department of Software Convergence

경희대학교
KYUNG HEE UNIVERSITY

Contents

- Docker overview
- ✴ Install Docker Desktop on your Notebook
- ✴ Test-drive of examples

Reading Articles

- Get Started: Docker overview

  https://docs.docker.com/get-started/docker-overview/

- Get Started: Part 1. Containerize an application

  https://docs.docker.com/get-started/workshop/02_our_app/

- Get Started: Part 2. Update the application

  https://docs.docker.com/get-started/workshop/03_updating_app/

Docker Pricing

Products Support Pricing Blog Docs Sign In

Yearly Monthly

Docker Personal	Docker Pro	MOST POPULAR Docker Team	Docker Business
$0	$11 per user/month	$16 per user/month	$24 per user/month
For individual developers who need the essential tools to build and deploy containers.	For individual professionals who require more advanced features and additional resources.	For small teams that need collaborative tools to make working together more efficient.	For enterprises desiring robust security, control, and compliance features.
Get started	Buy now	Buy now	Buy now Contact sales

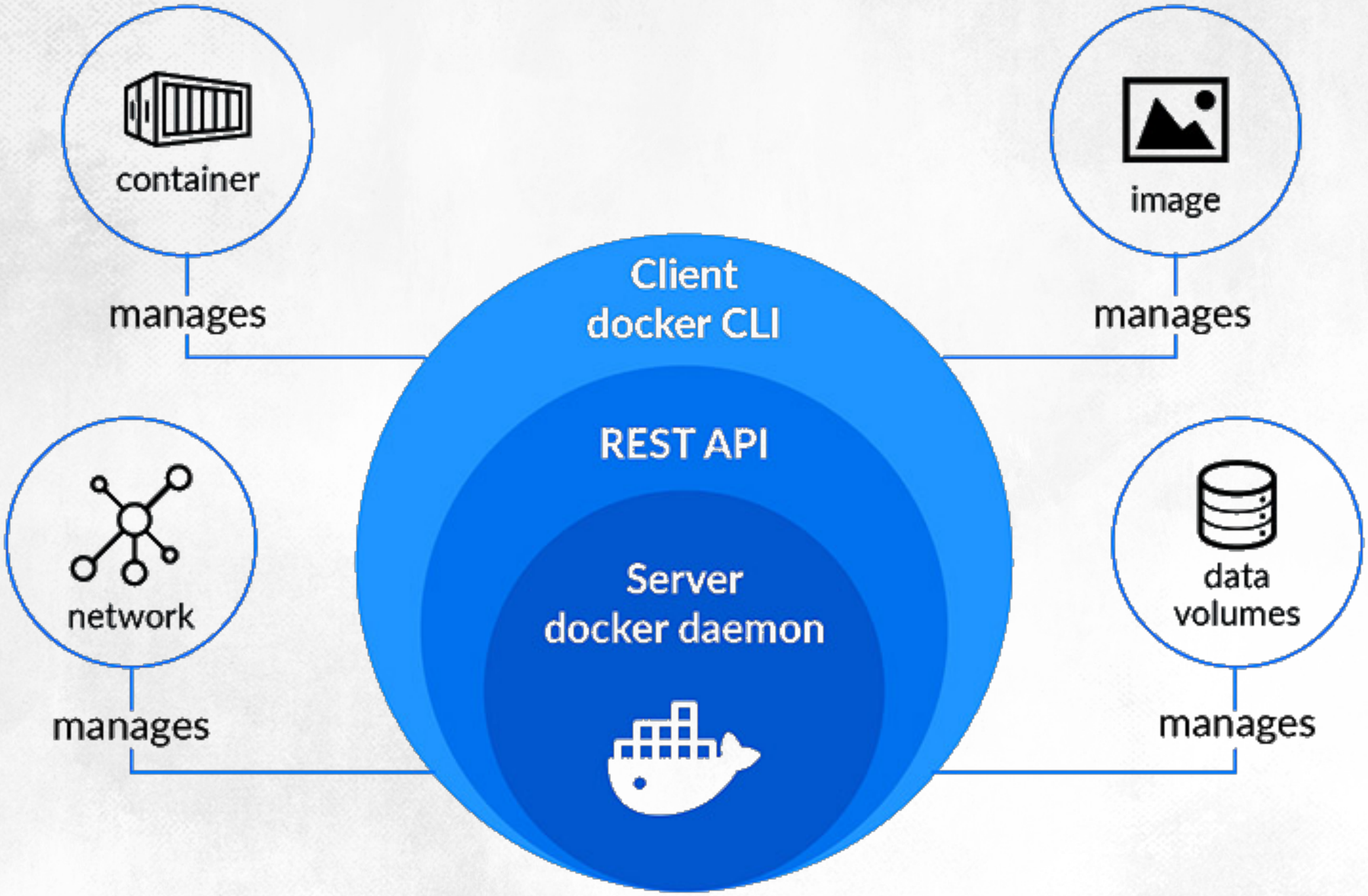
Reference: https://www.docker.com/pricing/

Docker Engine

- A persistent process that manages containers
- It does the core work of Docker
  - Building, running, & distributing containers
- It is running as a background service or daemon on the host

Reference: https://docs.docker.com/engine/docker-overview/

Docker Engine



Reference: https://dzone.com/articles/how-do-the-docker-client-and-docker-serves-work

Docker Daemon

- Listens for Docker API requests and manages Docker objects
  - e.g., images, containers, networks, & volumes
- Communicate with other daemons to manage Docker services

Reference: https://docs.docker.com/engine/docker-overview/

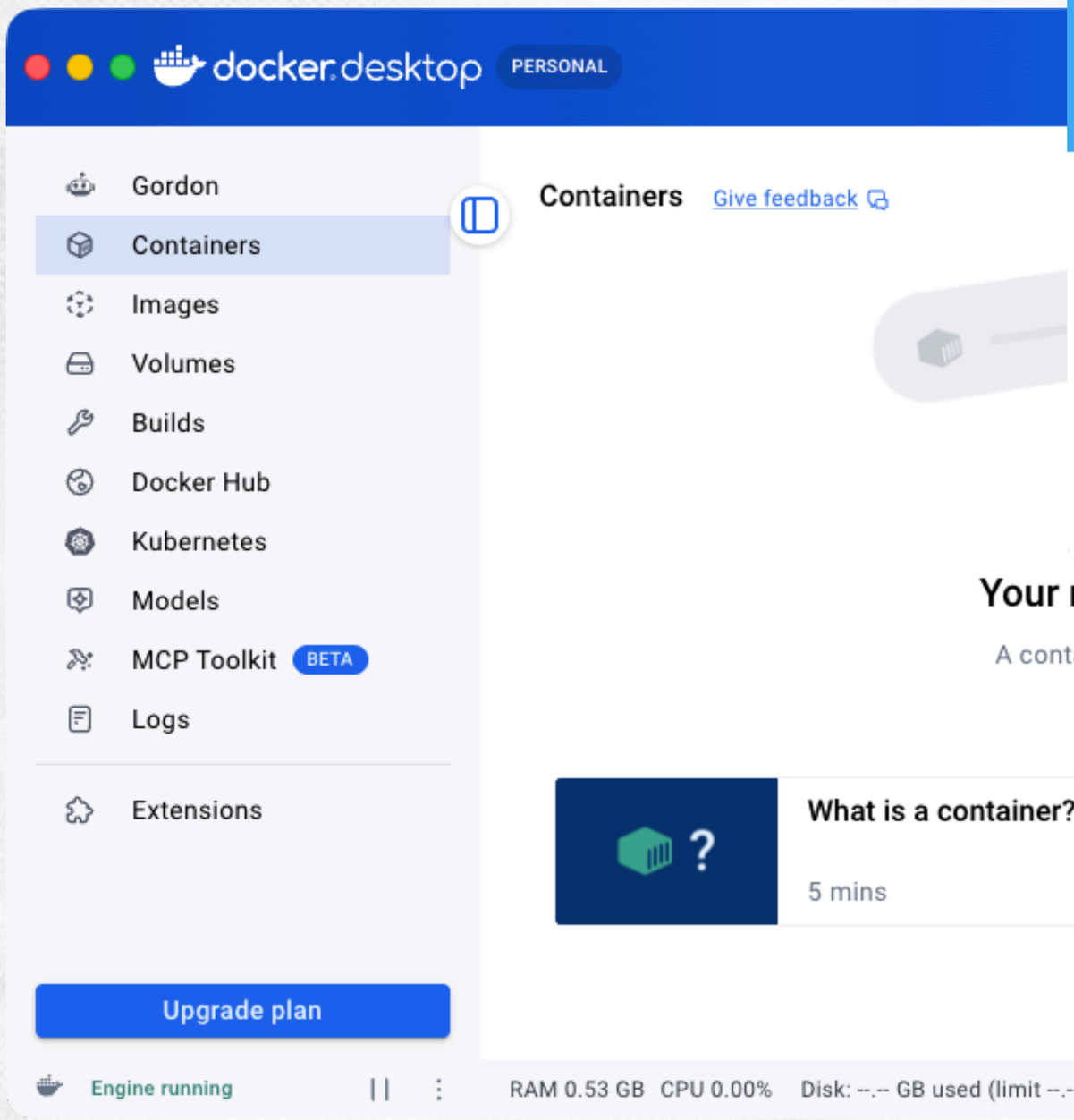
Docker Client

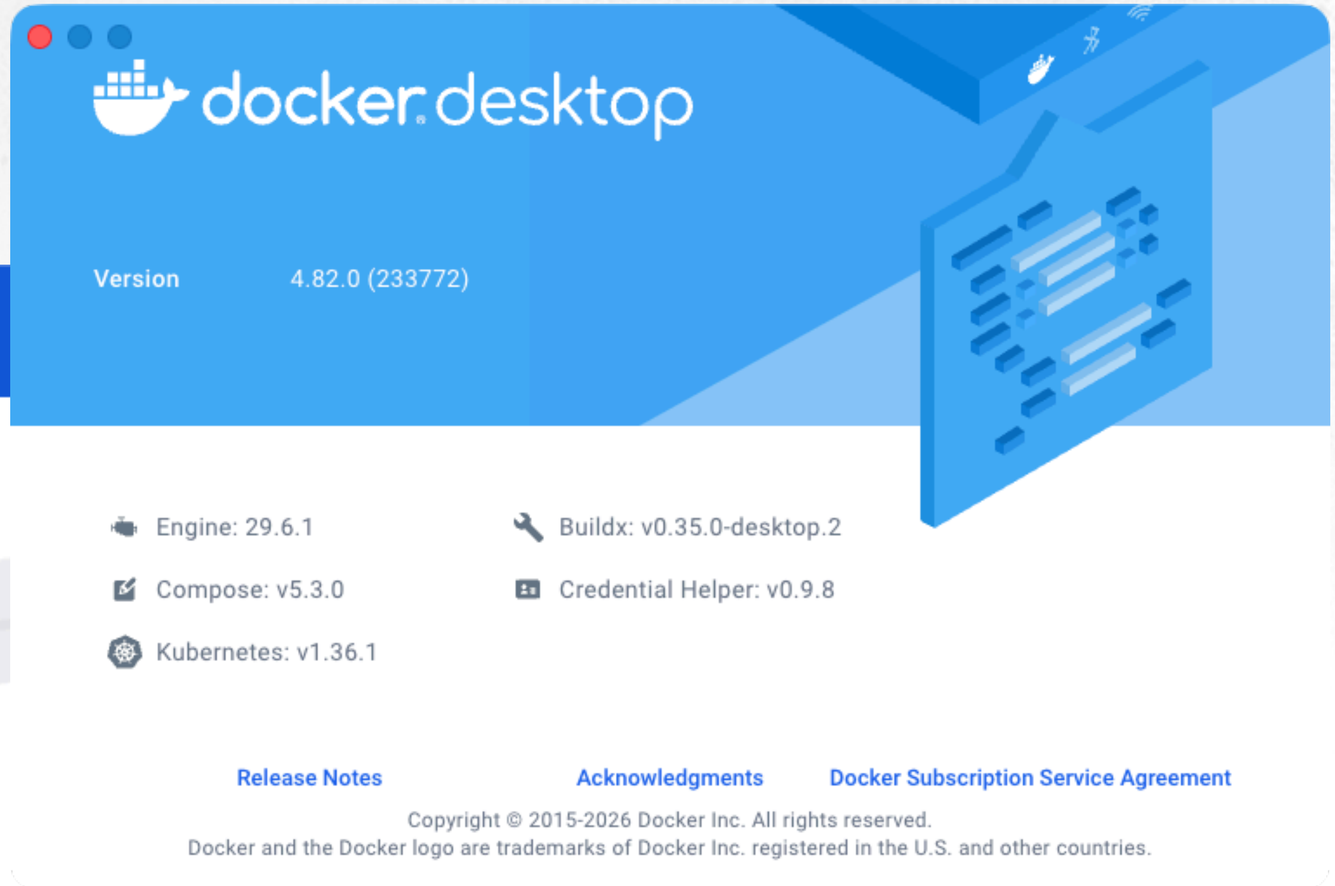
- A command line interface (CLI) used to give instructions to the Docker engine
- Similar to the client-server architecture of the Web
  - A client system uses an interface (e.g., web browser) to interact with a remote server.
- Docker client talks to the Docker engine that performs the work of containers and containerization
  - e.g., Tell the Docker daemon to pull an image from a registry
- The docker command uses the Docker API
- The Docker client can communicate with more than one daemon

Reference: https://docs.docker.com/engine/docker-overview/

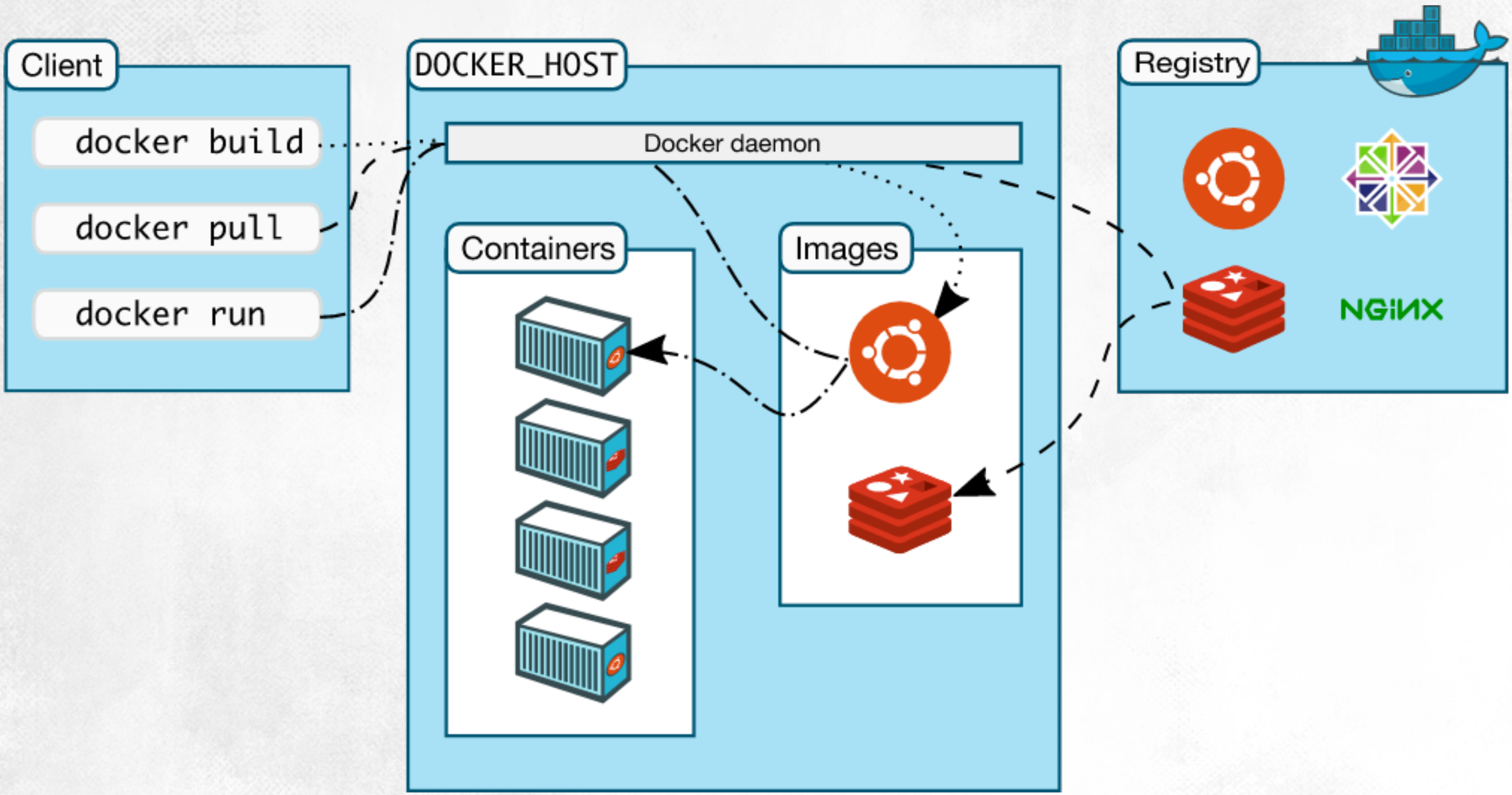
Docker Desktop Dashboard

- GUI Interface





Docker Architecture



Reference: https://docs.docker.com/engine/docker-overview/

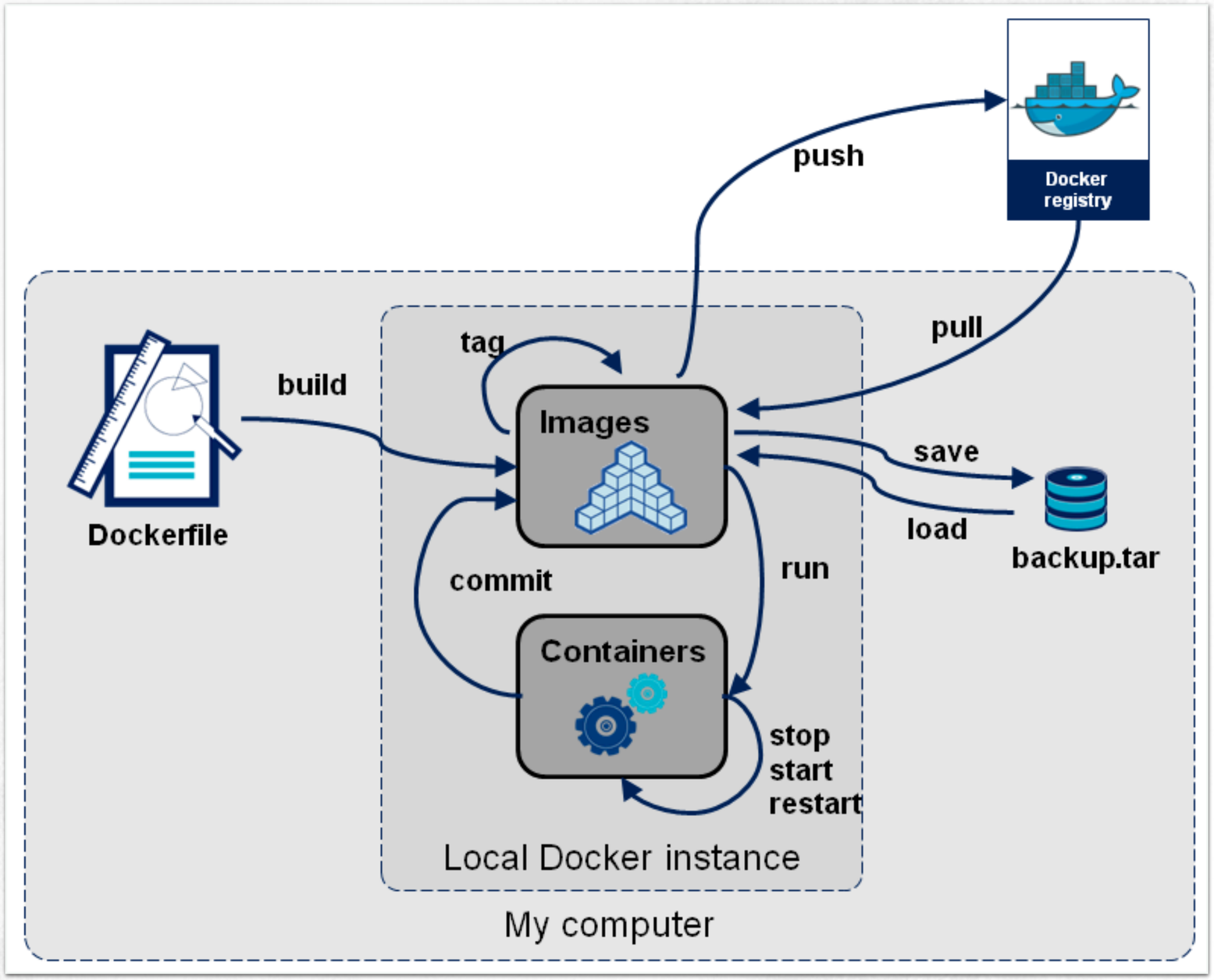
Docker Registries

- Store Docker images
- Public registry
  - Anyone can use
  - Docker Hub & Docker Cloud
- Private registry
  - Docker Trusted Registry (DTR)
- Docker is configured to look for images on Docker Hub by default

Reference: https://docs.docker.com/engine/docker-overview/

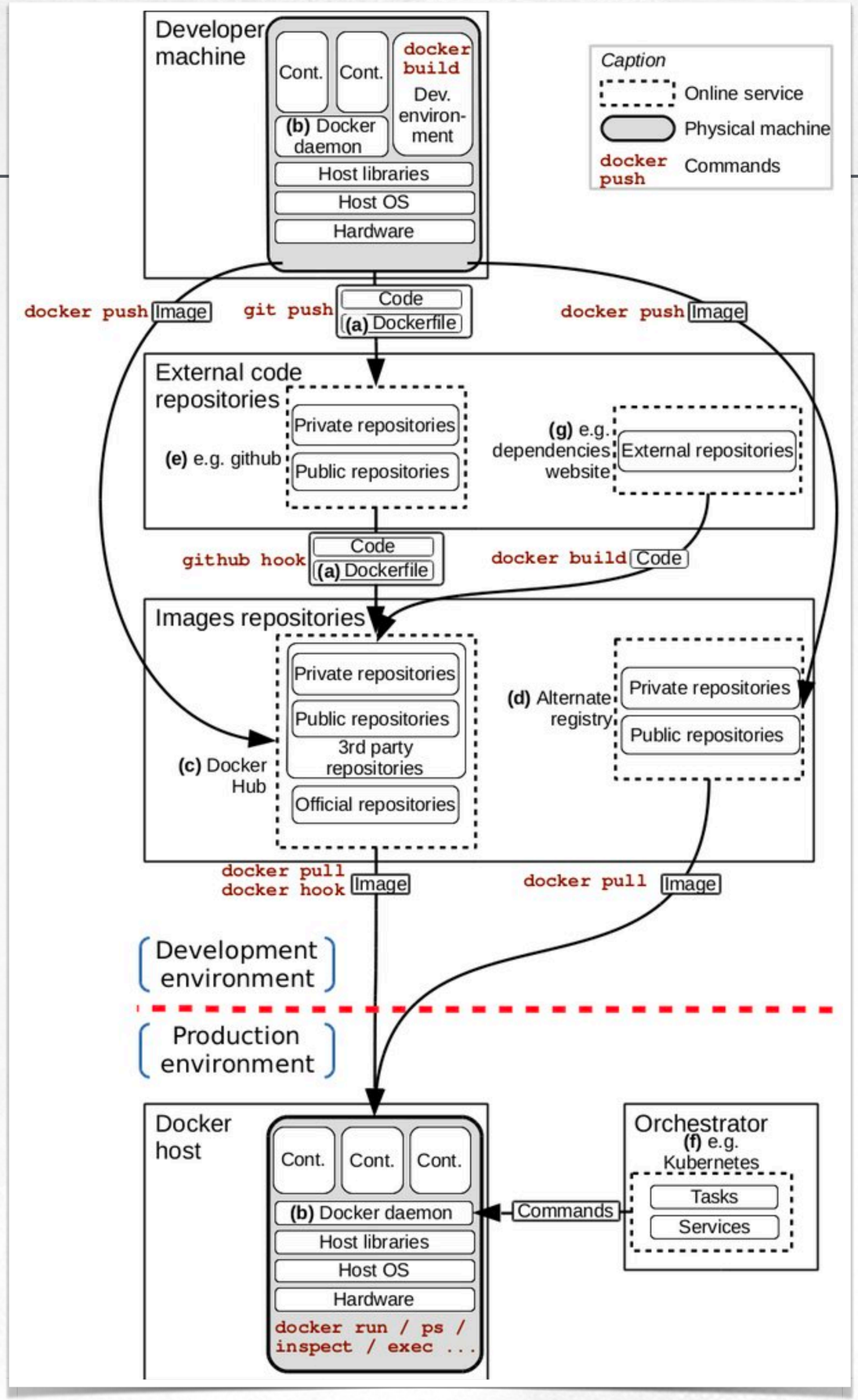
Docker Registries

- Pulling required images from a configured registry
  - docker pull
  - docker run
- Pushing an image to a configured registry
  - docker push



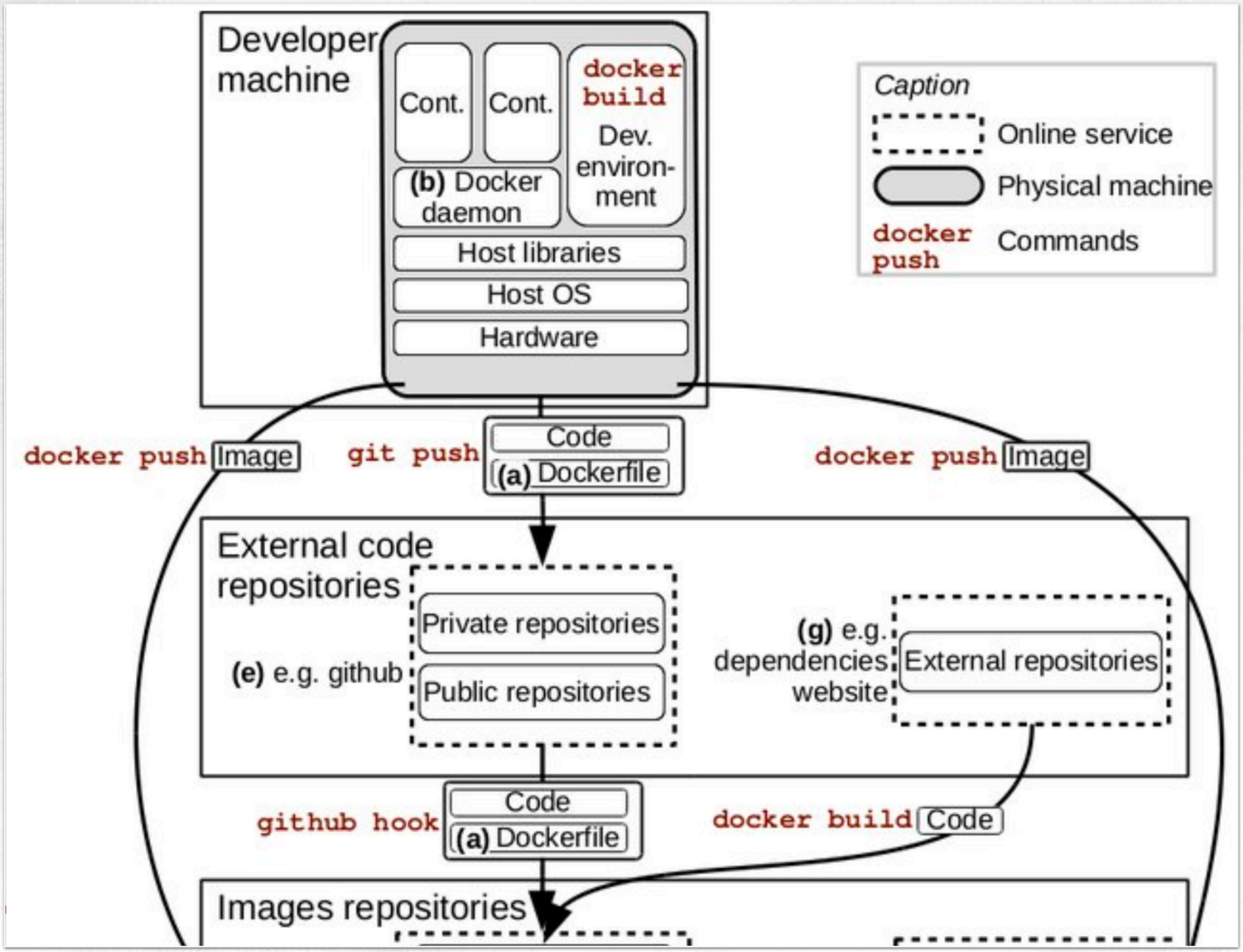
Reference: https://docs.docker.com/engine/docker-overview/

Docker Ecosystem



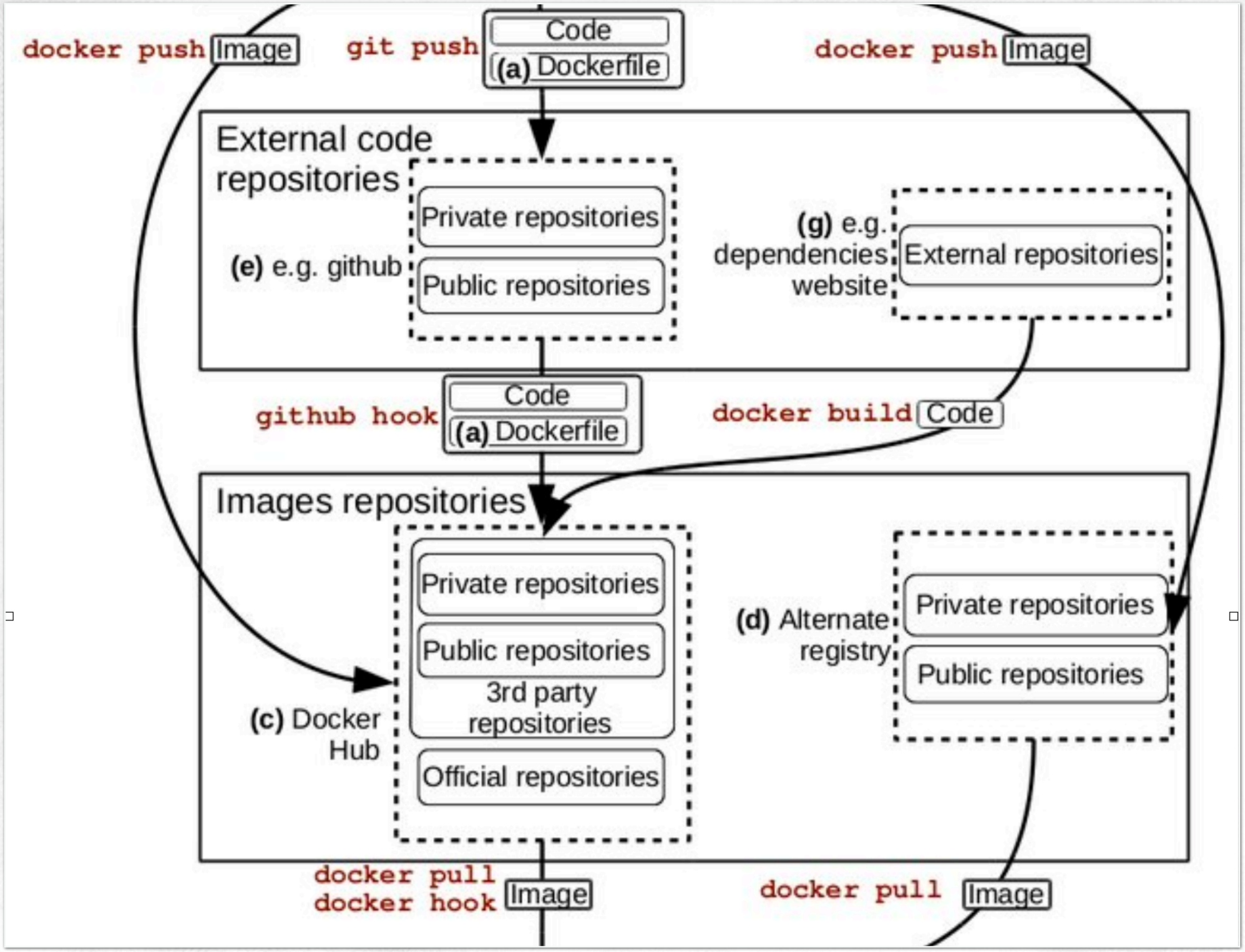
Reference: Source: Combe, T., Martin, A. and Di Pietro, R., 2016. To Docker or Not to Docker: A Security Perspective. *IEEE Cloud Computing*, *3*(5), pp.54-62.

Docker Ecosystem: Development (1/2)



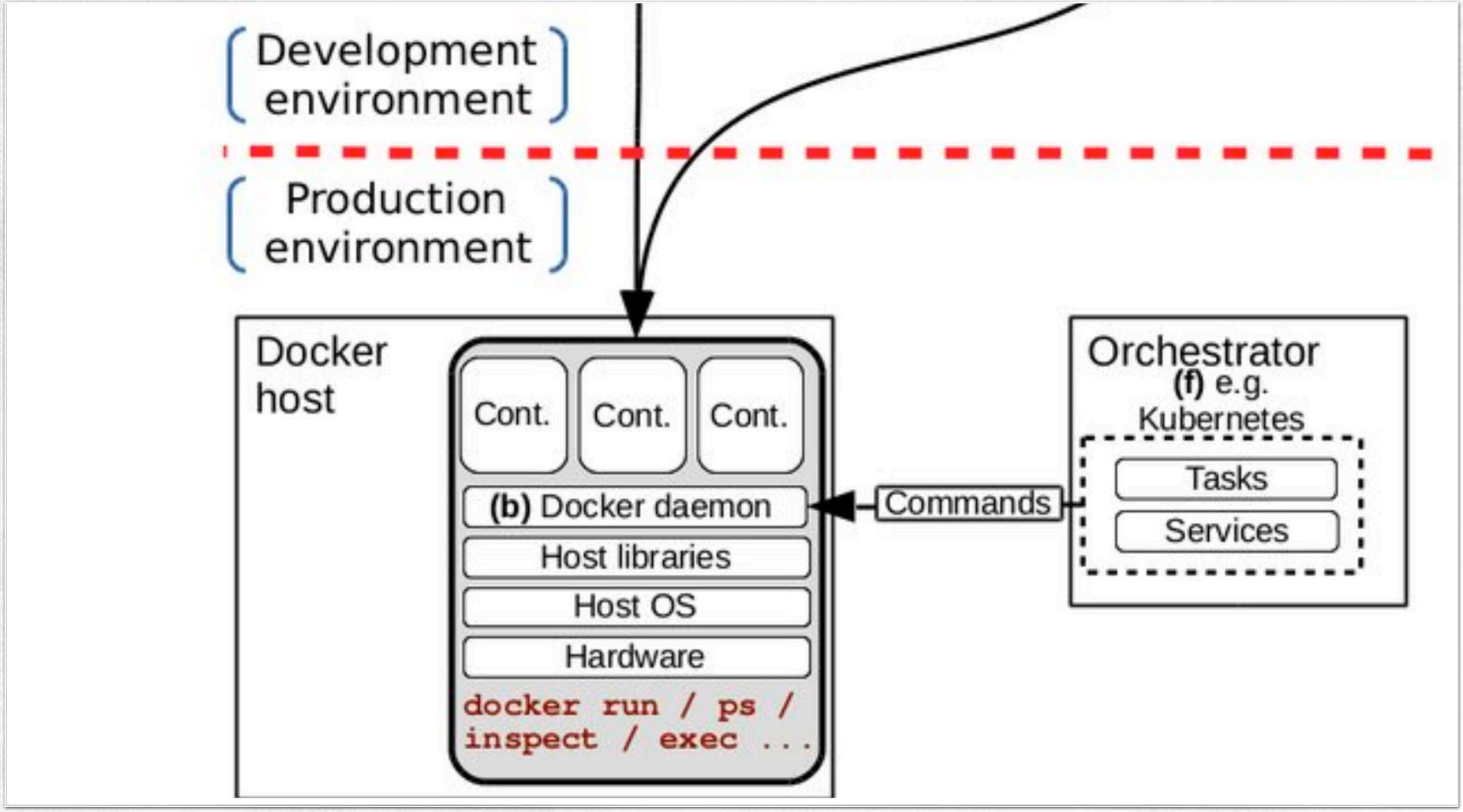
Reference: Source: Combe, T., Martin, A. and Di Pietro, R., 2016. To Docker or Not to Docker: A Security Perspective. *IEEE Cloud Computing*, *3*(5), pp.54-62.

Docker Ecosystem: Development (2/2)



Reference: Source: Combe, T., Martin, A. and Di Pietro, R., 2016. To Docker or Not to Docker: A Security Perspective. *IEEE Cloud Computing*, *3*(5), pp.54-62.

Docker Ecosystem : Production



Reference: Source: Combe, T., Martin, A. and Di Pietro, R., 2016. To Docker or Not to Docker: A Security Perspective. *IEEE Cloud Computing*, *3*(5), pp.54-62.

Underlying Technologies

- Written in Go language
- Takes advantages of features of the Linux kernel
  - Namespaces
  - Control groups
  - Union file systems
  - Container format

Reference: https://docs.docker.com/engine/docker-overview/

Namespaces

- Docker uses a technology called namespaces to provide the isolated workspace called the container.
- When you run a container, Docker creates a set of namespaces for that container.
- These namespaces provide a layer of isolation.
- Each aspect of a container runs in a separate namespace and its access is limited to that namespace.

- The `pid` **namespace:** Process isolation (PID: Process ID).
- The `net` **namespace:** Managing network interfaces (NET: Networking).
- The `ipc` **namespace:** Managing access to IPC resources (IPC: InterProcess Communication).
- The `mnt` **namespace:** Managing filesystem mount points (MNT: Mount).
- The `uts` **namespace:** Isolating kernel and version identifiers. (UTS: Unix Timesharing System).

Reference: https://docs.docker.com/engine/docker-overview/

Control Groups (cgroups)

- Responsible for resource management
- A cgroup limits an application to a specific set of resources
- Control groups allow Docker Engine to share available hardware resources to containers and optionally enforce limits and constraints
  - e.g., you can limit the memory available to a specific container

Reference: https://docs.docker.com/engine/docker-overview/

Installation

- Docker - Download and Install

  https://docs.docker.com/get-docker/



🌟HOMEWORK🌟

Test-Drive {from Lecture.5}

docker run hello-world

Install Ubuntu (Docker Way)

Install Ubuntu - Full Script (Docker Way)

Install Alpine - Full Script (Docker Way)

Install Python - Full Script (Docker Way)

Test-Drive

- Get Started: Part 1. Containerize an application

  https://docs.docker.com/get-started/workshop/02_our_app/

- Get Started: Part 2. Update the application

  https://docs.docker.com/get-started/workshop/03_updating_app/

Thank you

경희대학교
KYUNG HEE UNIVERSITY